Executive Summary

The `stPlatformAttitude` module converts a star tracker attitude measurement from the star tracker platform frame into the spacecraft body frame. It takes the inertial-to-platform attitude quaternion and platform-frame angular velocity quaternion from the star tracker platform solution (`StPlatformAttitudeMsgF32Payload`) and converts them using a configured mounting MRP.

The output is the inertial-to-body MRP $\boldsymbol{\sigma}_{B/N}$ and the body-frame angular velocity ${}^{\mathcal{B}}\boldsymbol{\omega}_{B/N}$, for downstream navigation and control modules. The sensor is rigidly mounted, so the angular velocity is only rotated into body components.

All numeric calculation is single-precision (FP32). The module has one algorithm and two interface adapters. A `SysModel` adapter connects it to the Xmera system with messages. A C shim connects it to the Adamant system through the C/Ada FFI.

Module Architecture

The module is split into a thin adapter that handles framework integration and an algorithm class that contains the pure math.

The algorithm class is `StPlatformAttitudeAlgorithm`. It keeps no cycle-to-cycle measurement state. `setConfig()` computes the inverse mounting MRP (`sigma_BP`) and the platform-to-body DCM (`dcm_BP`) once per configuration change.

Adapter Layer

The adapter class is `StPlatformAttitude`. It inherits from `SysModel` and owns the input and output messages. Each `updateState()` copies `platformAttitude` and `platformAngularVelocity` into `q_PN` and `rateQuaternion_PN` without conversion or reordering, passes them to the algorithm, and publishes `sigma_BN` and `omega_BN_B` with `solutionTimestamp` as the time tag in seconds (`double`). The output message is written every cycle. If the input message has not been written, zero quaternions and a zero time tag are used, so the output is `sigma_BP` with zero angular velocity and a zero time tag.

Table 1: Module I/O Messages

Msg Variable Name	Msg Type	Description
`stSensorInMsg`	`StPlatformAttitudeMsgF32Payload`	Star tracker platform attitude solution: attitude quaternion, rate quaternion and time tag.
`stAttOutMsg`	`STAttMsgF32Payload`	Body-frame attitude output: inertial-to-body MRP, body-frame angular velocity and the time tag passed through.

Configuration

`sigma_PB` is validated in `StPlatformAttitudeConfig::create()` when the adapter's `reset()` or `reconfigure()` runs; the algorithm never sees an invalid configuration.

Table 2: Configuration parameters

Parameter Name	Type	Units	Default	Description
`sigma_PB`	`Eigen::Vector3f`	[-]	Zero	MRP of the platform frame relative to the body frame. Must be finite. A shadow set is stored as its principal set. The default (zero) assumes the star tracker platform frame is aligned with the spacecraft body frame; set the actual mounting MRP otherwise.

Two-Phase Initialization

The adapter is set up in two phases. First, create the module, set `sigma_PB`, connect `stSensorInMsg`, and add the module to the simulation task; nothing is checked in this phase and the algorithm is not yet built. Second, `reset()`, called by `InitializeSimulation()`, checks that `stSensorInMsg` is connected, validates `sigma_PB`, and builds the algorithm. It throws if either check fails:

```
module = stPlatformAttitudeF32.StPlatformAttitude()
module.modelTag = "stPlatformToBody"
module.sigma_PB = sigma_PB
module.stSensorInMsg.subscribeTo(stSensorMsg)

scSim.AddModelToTask(simTaskName, module)
scSim.InitializeSimulation()  # runs reset()
```

After `reset()`, `updateState()` runs each cycle. To change the mounting, set `sigma_PB` and then call `reconfigure()`; the new value is used from the next `updateState()`. If `updateState()` or `reconfigure()` is called before `reset()`, an `XmeraLifecycleException` is raised.

Mathematical Formulation

The module works with three reference frames: the inertial frame $\mathcal{N}$, the spacecraft body frame $\mathcal{B}$, and the star tracker platform frame $\mathcal{P}$.

Inertial-to-Platform Attitude

The star tracker gives the inertial-to-platform attitude as a scalar-last quaternion $\mathbf{q}_{P/N} = [q_0, q_1, q_2, q_3]^T$, with q_3 the scalar part. It is reordered to the scalar-first Euler parameter set

$$\boldsymbol{\beta}_{P/N} = [q_3, q_0, q_1, q_2]^T$$

and converted to the equivalent MRP set $\boldsymbol{\sigma}_{P/N}$ (`epToMrp`).

Inertial-to-Body Attitude

The platform-to-body MRP is the inverse of the configured mounting MRP,

$$\boldsymbol{\sigma}_{B/P} = -\boldsymbol{\sigma}_{P/B}.$$

The inertial-to-body MRP is the composition of the inertial-to-platform rotation followed by the platform-to-body rotation, $[BN] = [BP][PN]$, computed by MRP addition (`addMrp`):

$$\boldsymbol{\sigma}_{B/N} = \boldsymbol{\sigma}_{B/P} \oplus \boldsymbol{\sigma}_{P/N}.$$

Angular Velocity from the Rate Quaternion

The star tracker does not report the angular velocity as a 3-vector. It encodes the rate as a rotation: the principal rotation vector (PRV) whose principal angle is the rate magnitude and whose principal axis is the rate direction,

$$\Phi = \|{}^{\mathcal{P}}\boldsymbol{\omega}_{P/N}\|, \qquad \hat{\mathbf{e}} = \frac{{}^{\mathcal{P}}\boldsymbol{\omega}_{P/N}}{\|{}^{\mathcal{P}}\boldsymbol{\omega}_{P/N}\|},$$

and sends that rotation as a scalar-last quaternion,

$$\mathbf{q}_{\omega} = \left[\hat{\mathbf{e}}\sin\!\left(\tfrac{\Phi}{2}\right),\ \cos\!\left(\tfrac{\Phi}{2}\right)\right] = [q_{\omega,0},\ q_{\omega,1},\ q_{\omega,2},\ q_{\omega,3}].$$

The angle Φ has the numerical value of the rate in rad/s. `rateQuaternion_PN` is therefore a rotation representation that carries a rate. It is not a quaternion derivative and not the rotation accumulated over a sample interval, so no time step is involved and nothing is divided by Δt.

The module converts this rotation back to its PRV and reads the PRV as the angular velocity. With $\mathbf{q}_{\omega,v} = [q_{\omega,0},\ q_{\omega,1},\ q_{\omega,2}]^T$, the principal angle and axis are

$$\Phi = 2\,\mathrm{atan2}(\|\mathbf{q}_{\omega,v}\|,\ q_{\omega,3})\,, \qquad \hat{\mathbf{e}} = \frac{\mathbf{q}_{\omega,v}}{\|\mathbf{q}_{\omega,v}\|},$$

and combining them gives

$${}^{\mathcal{P}}\boldsymbol{\omega}_{P/N} = \Phi\,\hat{\mathbf{e}} = \frac{2\,\mathrm{atan2}(\|\mathbf{q}_{\omega,v}\|,\ q_{\omega,3})}{\|\mathbf{q}_{\omega,v}\|}\,\mathbf{q}_{\omega,v}.$$

In the code, the rate quaternion is reordered to scalar-first and passed to `epToPrv` from `rigidBodyKinematics.hpp`, which computes Φ and $\hat{\mathbf{e}}$ this way. When $\|\mathbf{q}_{\omega,v}\| = 0$ the angular velocity is set to zero, avoiding a divide-by-zero.

A common way to invert this packing uses the arccosine of the scalar part,

$$k = \frac{2\arccos\left(q_{\omega,3}\right)}{\sqrt{q_{\omega,0}^2 + q_{\omega,1}^2 + q_{\omega,2}^2}}, \qquad {}^{\mathcal{P}}\boldsymbol{\omega}_{P/N} = k\,\mathbf{q}_{\omega,v}.$$

At small rates the scalar part is close to 1, where the arccosine loses float32 precision. To avoid that, the module uses the four-quadrant arctangent shown above.

Body-Frame Angular Velocity

Because the sensor is rigidly mounted, $\boldsymbol{\omega}_{B/N} = \boldsymbol{\omega}_{P/N}$, so only the components change. The platform-frame components are transformed into body-frame components with the transpose of the mounting DCM, $[BP] = [PB]^T$, where $[PB]$ is the DCM of the configured mounting MRP $\boldsymbol{\sigma}_{P/B}$:

$$^{\mathcal{B}}\boldsymbol{\omega}_{B/N} = [BP]\,{}^{\mathcal{P}}\boldsymbol{\omega}_{P/N}.$$

Assumptions and Limitations

- The star tracker platform is rigidly mounted to the hub, so $\boldsymbol{\omega}_{B/N} = \boldsymbol{\omega}_{P/N}$ and `sigma_PB` is constant between reconfigurations.
- `q_PN` must have length 1, as every attitude quaternion does. The sensor interface defines it as unit length, so the module does not check or renormalize it.
- If the input message has not been written, the output is not a valid measurement and carries a zero time tag (see Adapter Layer).